Recommendation engine: `engine/recommend.py`

Turns per-VM statistics (`stats.py`) into actions: resize, stop, schedule, upsize, delete or release, each with a saving, a risk level and a confidence score. Every card carries its own `maths` lines (the formula with real numbers), shown in the UI under "Show the maths".

1. What it does

Inputs: a VM's inventory record, its statistics, and its monthly cost breakdown. **Outputs:** a list of recommendation cards per VM (one VM can get a resize and a schedule), plus one card per qualifying orphan volume or IP.

2. Order of checks (in `build.py`, before `recommend.py`)

For every VM, the first rule that matches decides:

1. **Not enough data:** more than 20% of readings missing, or under 2 days of data → "review" card, HIGH risk, confidence 20, no change suggested (the peak could be in the gap).
2. **Active anomaly** (from `alerts.py`) → "investigate" card, sizing paused, HIGH risk, confidence 30. Sizing on incident data would "learn" the incident and recommend an upsize.
3. Everything else goes to `recommend_vm`. If an uploaded VM reports less often than every 15 minutes, a resize is turned into a "review" card because short peaks cannot be seen.
4. If the data window is under 30 days, confidence drops 10 and risk rises one step on resize, schedule and stop cards.

3. Rules inside `recommend_vm` (first match wins, then schedule is added)

Rule 1: Idle

All must hold: p95 CPU < 5%, p95 network < 0.5 MB/s, p95 IOPS < 10, at least 14 days of data.

- **Production VM:** "review" card (HIGH risk). It is never stopped automatically; the owning team confirms.
- **Non-production VM:** "stop" card (LOW risk). Saving = the plan price (compute only; disk is still billed while stopped).
- Returns immediately; no resize or schedule on top.

Rule 2: Upsize

If p99 CPU or p99 memory is above 90% → "upsize" card (LOW risk). If the computed plan is not larger than the current one, the next plan with more vCPU and at least the current RAM is used. The "saving" is negative (extra cost, for reliability). Returns immediately.

Rule 3: Resize

Compute the smallest safe plan with `size_for` (section 4). If it is cheaper than the current plan → "resize" card, saving = current price − new price.

Rule 3b: Keep (peak protected)

If the safe plan is not cheaper and average CPU is below 20% → "keep" card (LOW risk, saving 0, not accepted by default). It shows what an average-based tool would have done: size for the average, then check the real peak on that plan. Example from a run: `prod-reports-01` averages 10.5%, so an average-based tool picks `s-2-8`; its p99 of 80.5% × 8 vCPU ÷ 2 = 322% CPU, so the VM would fall over. If the p99 alone would fit, the busiest single reading is shown instead, because that is what blocked the shrink.

Rule 4: Off-hours schedule (non-production only, added on top)

Conditions: environment is not prod, at most 6 hours must be kept on, and quiet off-window average CPU < 5%.

1. Busy night hours come from `stats.py` (off-window p95 CPU > 15%).
2. Keep set = busy hours plus the hour before each (1-hour warm-up).
3. Weekly hours: the off window is 12 h × 5 weekdays + 48 weekend hours = 108 of 168.
4. Each kept hour is on every weekend day (−2 weekly hours), and on weekdays too if it falls inside the 20:00–08:00 window (−5 more).
5. Fraction stopped = (108 − kept hours) ÷ 168.
6. Saving = price after any resize × fraction. The price after resize is used, so the two savings are not double counted. Storage is not counted as a saving.
7. Risk: LOW for dev, MEDIUM for staging.

A night job never blocks the schedule: the VM simply stays on for the job's hours.

Orphans (`recommend_orphan`)

Unattached for at least 7 days (measured to the end of the data window):

- **Volume:** snapshot then delete. Saving = GB × ₹6 − GB × ₹2 = GB × ₹4 per month. LOW risk.
- **Public IP:** release. Saving ₹300 per month. LOW risk.
- Confidence is fixed at 95 because this is an inventory fact, not a measurement.

4. Sizing formula (`size_for`)

Step	Quantity	Formula	Why
1	vCPU for the typical peak	current vCPU × p99 CPU % ÷ 70%	leaves 30% headroom at the p99 peak
2	vCPU for the worst spike	current vCPU × max CPU % ÷ 95%	the single busiest reading must stay under 95%
3	**Required vCPU**	the larger of steps 1 and 2	both limits must hold
4	RAM for the typical peak	current RAM × p99 memory % ÷ 75%	leaves 25% buffer at the p99 peak
5	RAM for the minimum margin	used GB + 1 GB (used GB = current RAM × p99 memory %)	at least 1 GB stays free
6	RAM for the worst spike	current RAM × max memory % ÷ 90%	out-of-memory kills processes, so the highest reading stays under 90%
7	**Required RAM**	the largest of steps 4, 5 and 6	all three limits must hold
8	**Chosen plan**	cheapest plan with vCPU ⩾ required vCPU and RAM ⩾ required RAM	if no plan fits, the largest plan is used

The 95% and 90% ceilings catch spikes shorter than the top 1% (about 15 minutes a day) that p99 cannot see.

Peak check on the new plan (shown on each card). The check divides today's load by the new plan's size:

Measure	Formula	Safe if
p99 CPU after	p99 CPU % × current vCPU ÷ new vCPU	⩽ 75%
Max CPU after	max CPU % × current vCPU ÷ new vCPU	⩽ 95%
p99 memory after	p99 memory % × current RAM ÷ new RAM	⩽ 80%
Max memory after	max memory % × current RAM ÷ new RAM	⩽ 90%

The resize is marked safe only if all four rows are within their limits.

5. Risk level

For resizes (`risk_level`):

- **HIGH** if p99 CPU after > 75%, or memory after > 75%, or the VM is memory-bound (p99 memory > 70%).
- **MEDIUM** if p99 CPU after ⩾ 60%, or memory after ⩾ 60%, or it is production, or it is

bursty (p99 ÷ p95 CPU > 2).

- **LOW** otherwise.

So any production resize is at least MEDIUM.

6. Risk score (0 to 1)

The label picks one third of the range; measured numbers pick the spot inside it:

Part	Formula
Risk score	band + ⅓ × (0.5 × limit pressure + 0.3 × environment weight + 0.2 × (1 − confidence))
Band	LOW = 0, MEDIUM = 1/3, HIGH = 2/3
Environment weight	dev = 0, staging = 0.5, prod = 1
Confidence	the card's confidence score ÷ 100

Limit pressure (0 far from a limit, 1 at it):

- resize: the highest of p99 CPU after ÷ 70%, p99 memory after ÷ 75%, max CPU after ÷ 95%, max memory after ÷ 90% (capped at 1)
- schedule: quiet off-hours CPU ÷ 5%
- stop: p95 CPU ÷ 5%
- orphans, upsizes, keep cards: 0

The label never changes, so any LOW change scores below any MEDIUM one. The score only breaks ties (used by the "hit my target" planner).

7. Confidence score

Start at 100, then subtract: missing readings (3 points per percentage point, capped at 30); fewer than 14 days of data (−20); erratic usage, CV > 1 (−15); short peaks seen, time above 80% between 0 and 3% (−10). Minimum 10, maximum 100. Each deduction is written on the card.

8. Cost used for savings

`monthly_cost` = plan price + disk GB × ₹6 + egress GB × ₹0.80 (+ ₹300 for an IP). Savings use plan price differences only, so disk and network are never claimed as savings for a stopped or resized VM.

9. Sample output (my run, seed 42)

Action	Count
resize	45
schedule	23
keep	12
delete_volume	10
stop	6
release_ip	6
upsize	4
review	2
investigate	1

Showcase VMs: `prod-reports-01` gets a "keep" card (322% overload on the average-based plan); `dev-search-04` gets a resize `s-8-32` → `m-2-16` saving ₹5,400 (MEDIUM, p99 CPU after 62%) plus a schedule saving ₹1,671 (LOW, computed on the resized ₹2,600 price); `prod-pay-api-07` gets an "investigate" card (p99 CPU 99% including the incident).

10. Assumptions

- CPU load scales inversely with vCPU count (same work, more cores), and memory use stays the same in GB.
- Prices are illustrative and equal in all regions.
- A change takes effect for a whole month at today's price; no reserved-instance discounts.
- A 30-day window is representative of normal behaviour.

11. Limitations

1. **Linear CPU scaling** ignores differences between plan families and noisy neighbours.
2. **One outlier can block a shrink**, because the 95% ceiling uses the single highest reading.
3. **Upsize fallback is lightly checked.** When the target plan is moved to a larger one, only the CPU p99 figure is recomputed and the card is marked safe without recomputing memory.
4. **Monthly jobs** may not appear in 30 days; confidence is lowered, but the engine cannot know about them.
5. **Limited interaction between cards.** A resize and a schedule on the same VM are netted (the schedule uses the resized price); other combinations of cards are not modelled.
6. **Humans decide.** Production idle VMs and anything needing investigation are never auto-accepted (`review`, `keep` and `investigate` cards default to not accepted).